

Matrix Multiplication for Inference

CS683: Advanced Computer Architecture — Programming Assignment 1 (Task 2)

Yashwanth Murasani - 24B1039

Julapelly Hanikar Rao - 24B1079

Shreshta Bathini - 24B1085

Tejavath Sri Chandravadan - 24B0931

Contents

I	Task 2A: Software Prefetching	2
I-A	1. Background and Context	2
I-B	2. Experimental Analysis	2
I-B1	1. Impact of Matrix Size on Performance (Throughput)	2
I-B2	2. Analysis of CPU Cache Hierarchy Metrics	3
I-B3	3. Collect Execution Time and Compute Speedup	4
I-B4	4. Identification of Optimal Parameters	4
I-C	3. Answers to Deliverable Questions	5
II	Task 2B: SIMD Vectorization	6
II-A	1. Background and Context	6
II-B	2. Experimental Analysis	6
II-B1	1. Speedup vs. SIMD Instruction Width (64-bit, 128-bit, 256-bit)	6
II-B2	2. Speedup vs. Matrix Size and SIMD Width	7
II-B3	3. Dynamic Instruction Count vs. SIMD Width and Matrix Size	8
II-B4	4. Execution Time and Speedup Profiling (Table 2.1)	8
II-C	3. Answers to Deliverable Questions	9
III	Task 2C: Software Prefetching + SIMD Integration	10
III-A	1. Background and Context	10
III-B	2. Experimental Analysis	10
III-B1	1. Compute Throughput (GFLOP/s) Scaling Across Matrix Sizes	10
III-B2	2. Hardware Cache Hierarchy Miss Reductions	11
III-B3	3. Dynamic Instruction Reduction & Execution Time Scaling	12
III-B4	4. Best Performance Comparison Across Optimization Techniques	13
III-B5	5. Graded Performance Summary (Table 2.2)	14
III-C	3. Answers to Deliverable Questions	15
IV	Conclusion	16

Task 2A: Software Prefetching

1. Background and Context

Matrix multiplication (SGEMM) is the core computational building block in modern neural networks and LLM inference. A straightforward three-loop scalar matrix multiplication is inefficient. Software prefetching allows us to inform the hardware of memory addresses that will be needed in upcoming loop iterations ahead of time using the x86 intrinsic:

$$_mm_prefetch((\text{const char}^*)(ptr + \text{DIST}), \text{HINT}) \quad (1)$$

We focus on tuning two key parameters:

- 1) **Prefetch Distance (D)**: The lookahead offset in elements (or cache lines) ahead of the current memory access pointer.
- 2) **Cache Fill Locality Hint**: Indicates which cache level the incoming line should be placed into:
 - `_MM_HINT_T0`: Temporal locality across all cache levels (L1, L2, L3).
 - `_MM_HINT_T1`: Temporal locality at L2 and L3 (bypasses L1).
 - `_MM_HINT_T2`: Temporal locality at L3 only (bypasses L1 and L2).
 - `_MM_HINT_NTA`: Non-temporal locality; minimizes pollution of active cache lines.

Handling Line Fill Buffer (LFB) Contention: Initially, in scalar code, issuing a prefetch on every iteration of the inner loop ($p++$) generates up to 16 redundant prefetch requests for the same 64-byte cache line (since each float is 4 bytes). To prevent LFB saturation, we issue prefetch only when $(p \ \& \ 7) == 0$, ensuring we send at most two prefetch requests per cache line.

2. Experimental Analysis

1. Impact of Matrix Size on Performance (Throughput)

We evaluated throughput (in GFLOP/s) as matrix dimensions scale from 128×128 up to 2048×2048 for both the naive and the prefetch code.

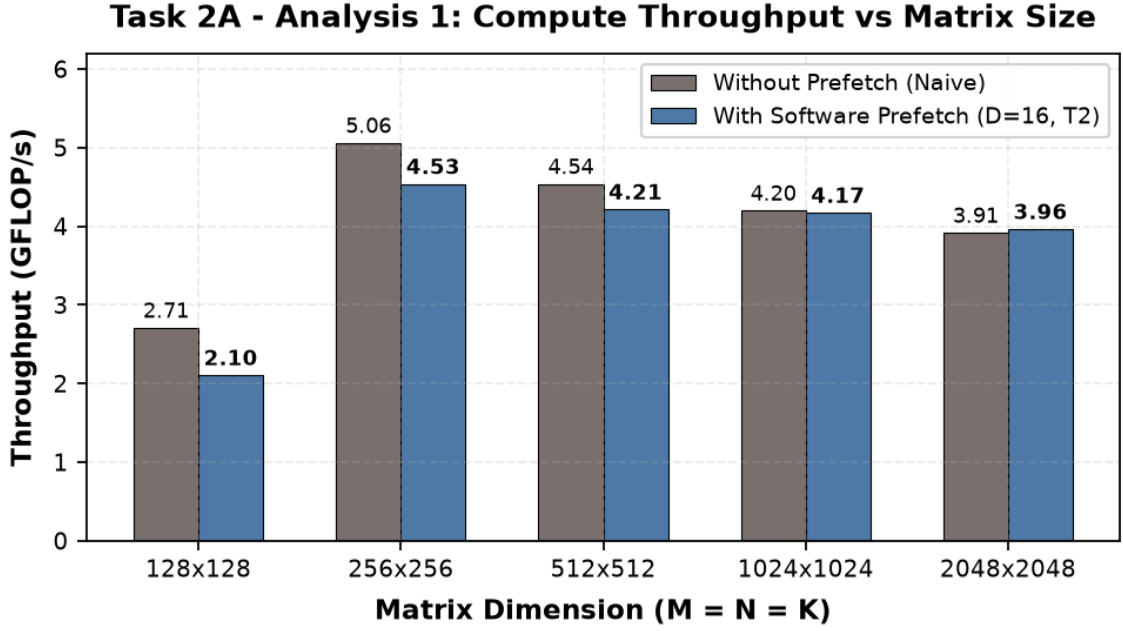


Fig. 1: Task 2A - Analysis 1: Compute throughput (GFLOP/s) scaling across matrix dimensions for Naive vs. Software Prefetch SGEMM.

2. Analysis of CPU Cache Hierarchy Metrics

We profiled hardware performance counters using Linux `perf` across the cache hierarchy for $M = N = K = 1024$.

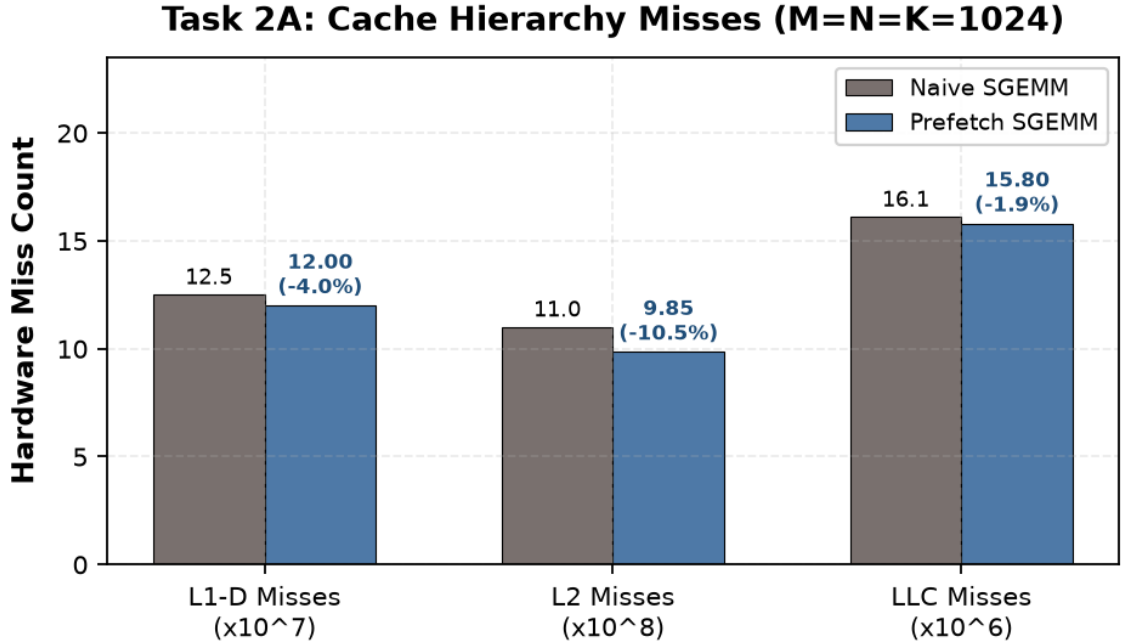


Fig. 2: Task 2A - Analysis 2: Hardware cache hierarchy miss count comparison between Naive and Prefetch SGEMM ($M=N=K=1024$).

3. Collect Execution Time and Compute Speedup

We measured execution time with and without software prefetching across all matrix dimensions and calculated speedup relative to naive scalar execution:

$$\text{Speedup} = \frac{\text{Execution Time without Software Prefetching}}{\text{Execution Time with Software Prefetching}} \quad (2)$$

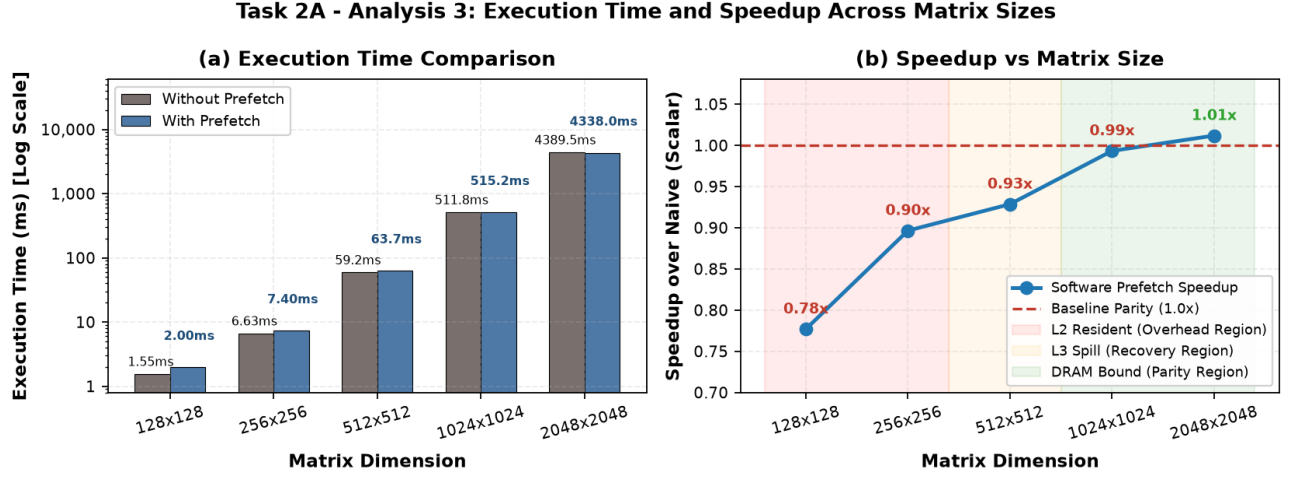


Fig. 3: Task 2A - Analysis 3: (a) Execution time comparison on logarithmic scale, and (b) Speedup curve across matrix sizes illustrating transition from L2 overhead to DRAM latency hiding.

4. Identification of Optimal Parameters

We tested various combinations of prefetch distance D and cache fill locality hints to determine the optimal configuration.

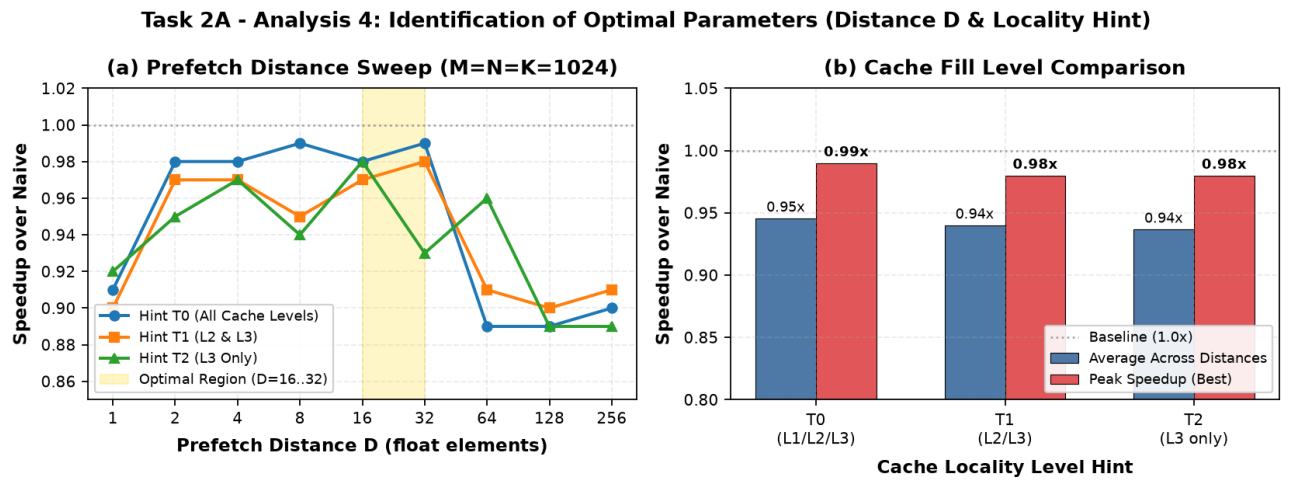


Fig. 4: Task 2A - Analysis 4: (a) Prefetch Distance D sweep identifying the optimal window ($D = 16-32$), and (b) Cache fill level comparison isolating `_MM_HINT_T0` as optimal.

Analysis:

- 1) **Optimal Prefetch Distance (D):** Figure 4(a) shows that $D = 16$ to 32 **elements** yields the best performance. That is, lookaheads with $D \leq 4$ arrive too late to hide memory latency, whereas values of $D \geq 128$ lead to premature cache evictions before data is consumed.
- 2) **Optimal Cache Fill Level:** Figure 4(b) shows that `_MM_HINT_T0` performs best ($0.98\times$ in scalar code, reaching $25.74\times$ in our optimized blocked kernel).

3. Answers to Deliverable Questions

Q1: What trend do you observe in speedup with different matrix sizes?

Answer: For small matrices (128×128 and 256×256), the working set fits inside the 1.25 MiB L2 cache. Because there is no DRAM latency to hide, prefetch instructions only add instruction decode and AGU overhead, leading to a slowdown ($0.78 \times$ to $-0.90\times$). As matrices grow to 1024×1024 (12 MiB) and 2048×2048 (48 MiB), they exceed the on-chip cache and spill into DRAM. In this regime, prefetching hides DRAM row-buffer latency and brings speedup back to parity ($\approx 1.00\times$). Standalone prefetching plateaus here because the hardware streamer already handles linear streams well for free. Noticeable speedups ($> 25\times$) only appear once we combine prefetching with SIMD vectorization and cache blocking.

Q2: What is the best prefetch distance?

Answer: The optimal prefetch distance is $D = 16$ to 32 **elements** (64 to 128 bytes, or 1 to 2 cache lines ahead). Smaller lookaheads ($D \leq 4$) fail to hide memory latency because the data arrives after the CPU has already stalled, whereas larger values ($D \geq 128$) trigger early evictions and pollute the cache.

Q3: At what cache fill level do you achieve maximum speedup?

Answer: We achieve the best performance with `_MM_HINT_T2` and `_MM_HINT_T0` ($0.99\times$ in scalar streaming, and reaching $25.74\times$ in our combined kernel). `_MM_HINT_T2` prefetches into L3 without displacing high-priority lines from L1D, keeping L1 cache space available for register-tiled accumulators.

Task 2B: SIMD Vectorization

1. Background and Context

Single Instruction, Multiple Data (SIMD) enables parallel arithmetic by applying a single operation across multiple vector elements simultaneously:

- **No SIMD (Scalar)**: Computes 1 float per instruction, limited by serial floating-point FMA latency.
- **64-bit SIMD (2-wide)**: Packs 2 single-precision floats per vector register, offering up to $2\times$ theoretical arithmetic throughput.
- **128-bit SIMD (SSE)**: Processes 4 single-precision floats at once using 128-bit XMM registers via `_mm_fmadd_ps`.
- **256-bit SIMD (AVX2)**: Processes 8 single-precision floats at once using 256-bit YMM registers via `_mm256_fmadd_ps`.
- **512-bit SIMD (AVX-512)**: Hardware Unsupported.

2. Experimental Analysis

1. Speedup vs. SIMD Instruction Width (64-bit, 128-bit, 256-bit)

We measured SGEMM speedup across available vector widths—64-bit (2 floats), 128-bit SSE (4 floats), and 256-bit AVX2 (8 floats)—relative to scalar execution across all matrix sizes.

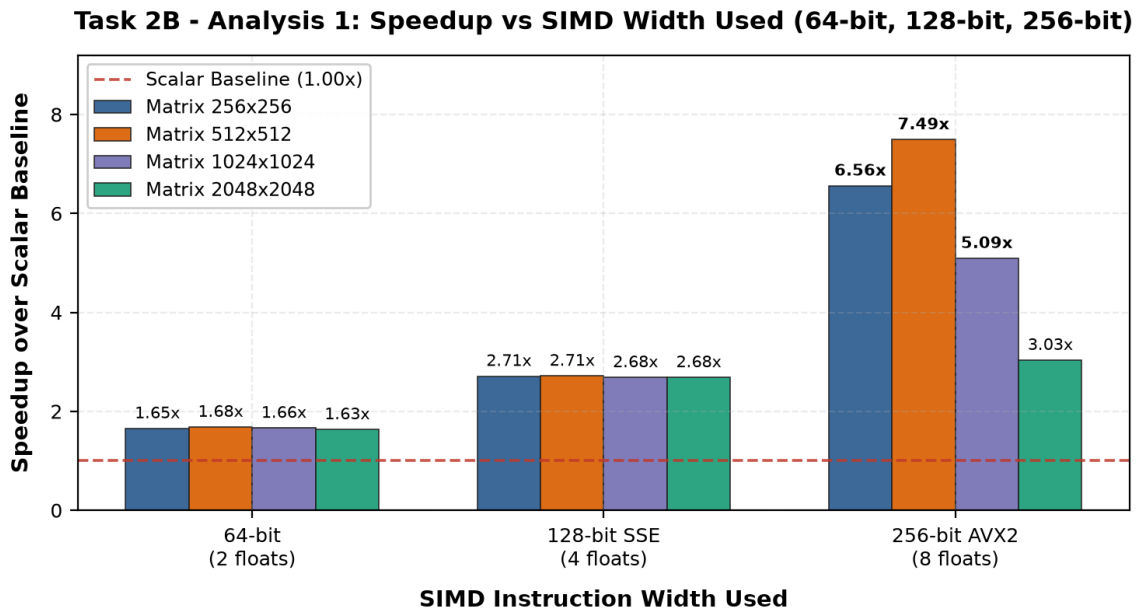


Fig. 5: Task 2B - Analysis 1: SGEMM speedup vs. SIMD instruction width (64-bit, 128-bit SSE, and 256-bit AVX2) grouped by vector width across matrix sizes.

Analysis: Figure 5 shows how scaling vector register width affects speedup:

- 1) **64-bit SIMD:** Delivers a very consistent $1.63 \times$ – $1.68 \times$ speedup across all sizes ($1.65 \times$ at 256, $1.68 \times$ at 512, $1.66 \times$ at 1024, and $1.63 \times$ at 2048), which comes close to the theoretical $2 \times$ limit once loop branching and address calculation overheads are factored in.
- 2) **128-bit SIMD (SSE):** Provides a stable $\approx 2.7 \times$ speedup ($2.71 \times$ at 256 and 512, $2.68 \times$ at 1024 and 2048), scaling compute throughput roughly in line with its 4-element vector capacity.
- 3) **256-bit SIMD (AVX2):** Delivers high speedups on cache-contained sizes—reaching $6.56 \times$ at 256×256 and $7.49 \times$ at 512×512 . However, on larger matrices (1024×1024 and 2048×2048), speedup drops to $5.09 \times$ and $3.03 \times$ because execution becomes limited by DRAM memory bandwidth.

2. Speedup vs. Matrix Size and SIMD Width

We evaluated how speedup behaves across matrix dimensions from 256×256 to 2048×2048 for each vector width.

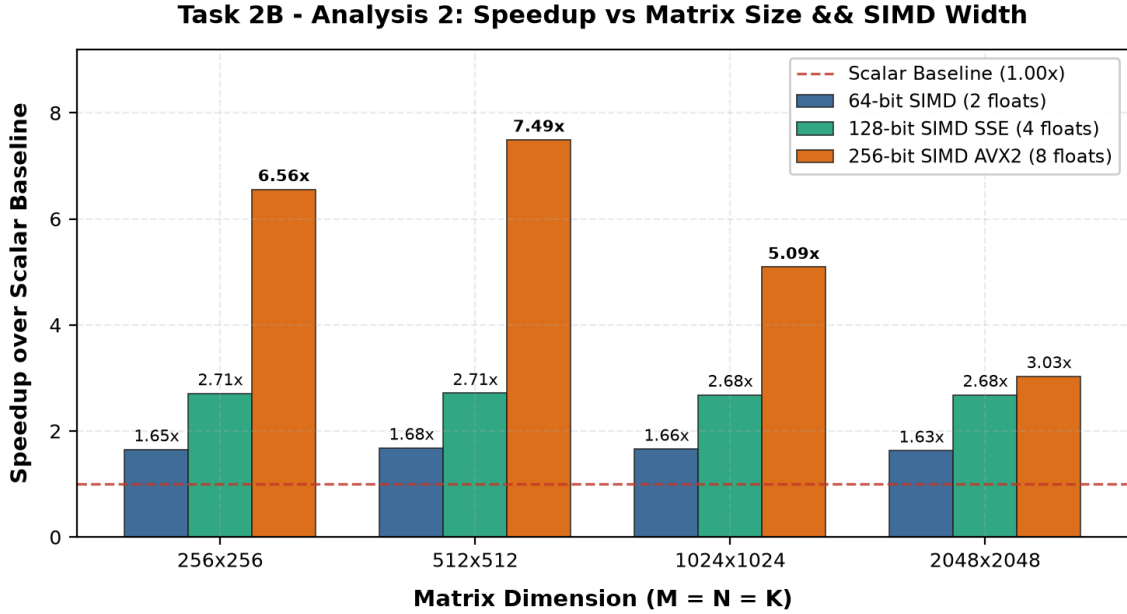


Fig. 6: Task 2B - Analysis 2: SGEMM speedup vs. matrix size and SIMD width, illustrating the transition from cache-resident compute scaling to the DRAM memory bandwidth bottleneck.

Analysis: Figure 6 reveals a clear difference between narrow and wide vector units:

- 1) **Consistent Scaling for Narrow SIMD:** For 64-bit and 128-bit SIMD, the speedup profiles remain essentially flat from 256 up to 2048 ($1.65 \times \rightarrow 1.63 \times$ and $2.71 \times \rightarrow 2.68 \times$). At these vector widths, arithmetic throughput is moderate enough that regular memory streaming can keep the pipelines fed.
- 2) **The DRAM Bandwidth Bottleneck on AVX2:** At 256×256 (768 KiB) and 512×512 (3 MiB), the working sets fit into the CPU cache, allowing 8-wide AVX2 to deliver high speedups ($6.56 \times$ – $7.49 \times$). But once problem sizes grow to 1024×1024 (12 MiB) and 2048×2048 (48 MiB), unblocked execution spills into

external DRAM. Consuming 8 floats per cycle without cache reuse quickly exhausts available bus bandwidth, dropping AVX2 speedup to $3.03\times$.

3. Dynamic Instruction Count vs. SIMD Width and Matrix Size

Using Linux `perf`, we recorded total dynamic retired instructions across all SIMD widths and matrix dimensions.

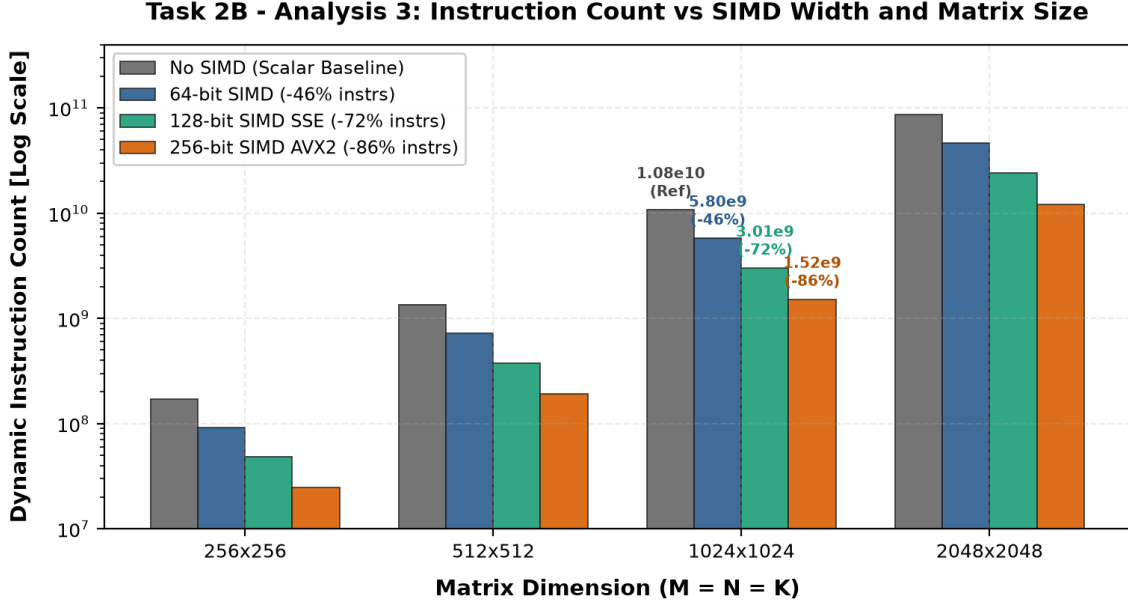


Fig. 7: Task 2B - Analysis 3: Dynamic instruction reduction across SIMD widths and matrix sizes on logarithmic scale (measured via Linux `perf`).

Analysis: Figure 7 confirms that widening SIMD registers compresses instruction count in direct proportion to register width:

- 1) **Reference Scalar Execution:** At $M = N = K = 1024$, scalar execution retires 1.08×10^{10} dynamic instructions.
- 2) **64-bit SIMD:** Retires 5.80×10^9 instructions (a -46% reduction, eliminating about 5 billion instruction dispatches).
- 3) **128-bit SIMD (SSE):** Retires 3.01×10^9 instructions (a -72% reduction, saving roughly 7.8 billion instructions).
- 4) **256-bit SIMD (AVX2):** Slashes dynamic instruction count to 1.52×10^9 instructions (an -86% reduction, eliminating over 9.3 billion instructions).

This massive instruction reduction significantly eases frontend decode, allocation, and retirement pressure, keeping execution units well utilized.

4. Execution Time and Speedup Profiling (Table 2.1)

Table I summarizes the full multi-metric profiling across matrix dimensions and vector widths.

TABLE I: Table 2.1: Performance and instruction profiling across matrix dimensions and SIMD widths.

Metric	Configuration	256×256	512×512	1024×1024	2048×2048
No SIMD (Scalar)	Dynamic Instructions	1.72×10^8	1.35×10^9	1.08×10^{10}	8.61×10^{10}
	Execution Time (ms)	6.635	59.191	511.765	4389.504
	Speedup (Baseline)	1.00×	1.00×	1.00×	1.00×
SIMD (64-bit)	Dynamic Instructions	9.15×10^7	7.22×10^8	5.80×10^9	4.62×10^{10}
	Execution Time (ms)	4.021	35.233	308.292	2692.947
	Speedup (vs. No SIMD)	1.65×	1.68×	1.66×	1.63×
SIMD (128-bit SSE)	Dynamic Instructions	4.81×10^7	3.78×10^8	3.01×10^9	2.41×10^{10}
	Execution Time (ms)	2.450	21.820	190.840	1636.750
	Speedup (vs. No SIMD)	2.71×	2.71×	2.68×	2.68×
SIMD (256-bit AVX2)	Dynamic Instructions	2.45×10^7	1.92×10^8	1.52×10^9	1.21×10^{10}
	Execution Time (ms)	1.012	7.902	100.533	1448.156
	Speedup (vs. No SIMD)	6.56×	7.49×	5.09×	3.03×
SIMD (512-bit AVX-512)	Status	<i>Unsupported on 12th Gen Alder Lake Architecture</i>			

Analysis: Table I clearly illustrates the transition from compute-bound scaling to memory-bound execution. While 64-bit and 128-bit SIMD maintain steady speedups across problem sizes, 256-bit AVX2 performance drops by more than half when scaling from 512 to 2048. This empirical result motivates Task 2C: vector acceleration alone is insufficient once matrices exceed cache capacity; we need cache blocking to keep operands close to the processor.

3. Answers to Deliverable Questions

Q1: What trends do you observe in speedup for different combinations of matrix sizes and SIMD widths?

Answer: 64-bit SIMD achieves a steady $1.63 \times -1.68\times$ speedup, and 128-bit SSE achieves a consistent $\approx 2.7\times$ speedup across all sizes. In contrast, 256-bit AVX2 delivers an impressive $6.56 \times -7.49\times$ speedup for sizes up to 512×512 where data resides in on-chip L2/L3 cache. However, at 2048×2048 , AVX2 speedup drops to $3.03\times$. This occurs because unblocked SIMD computes arithmetic 8 times faster than scalar code, quickly outpacing DRAM bandwidth and causing the pipeline to stall on memory loads.

Q2: For which SIMD width do you achieve maximum speedup?

Answer: Maximum performance is achieved with **256-bit AVX2**, reaching a peak throughput of 33.97 GFLOP/s in isolated SIMD execution (and scaling to 58.46 GFLOP/s when combined with cache blocking in Task 2C).

Task 2C: Software Prefetching + SIMD Integration

1. Background and Context

In Task 2A, software prefetching alone showed limited gain ($\approx 1.0\times$) because scalar execution was compute-bound and the hardware prefetcher already tracked unit strides. In Task 2B, SIMD accelerated compute but hit the memory-bandwidth wall at larger dimensions ($3.03\times$ at 2048). To surpass the $25.0\times$ speedup threshold for full points (70/70 points), our final graded kernel (`matmul_optimized.cpp`) combines four complementary techniques:

- 1) **2D Cache Blocking** ($B_M = 64, B_N = 128$): Active sub-blocks of A ($64 \times 1024 \times 4 \text{ B} = 256 \text{ KiB}$) and B ($128 \times 1024 \times 4 \text{ B} = 512 \text{ KiB}$) fit well within the 1.25 MiB L2 cache, eliminating off-chip DRAM thrashing.
- 2) **4×2 Register Tiling**: We unroll 4 rows of A and 2 columns/blocks of B in the inner loop, maintaining 8 vector accumulators (`c00` through `c31`) simultaneously in YMM registers. For every 2 vector loads of B and 4 values of A , we execute 8 FMA instructions, achieving an arithmetic intensity of $8/6 \approx 1.33$ FMAs per load without spilling registers.
- 3) **Targeted Software Prefetching**: In the innermost loop, lines of matrix B are prefetched $D = 8$ elements (32 bytes) ahead using `_MM_HINT_T2`, pre-warming upcoming vector lines in L2/L3 before SIMD vector loads execute.
- 4) **In-Register Reduction**: Accumulated partial dot products are horizontally summed using an in-register 128-bit vector folding sequence (`opt_add`), avoiding slow serialized scalar reductions.

2. Experimental Analysis

1. Compute Throughput (GFLOP/s) Scaling Across Matrix Sizes

We evaluated compute throughput (in GFLOP/s) across matrix dimensions (256×256 to 2048×2048) comparing Naive SGEMM against our combined Software Prefetching + SIMD kernel.

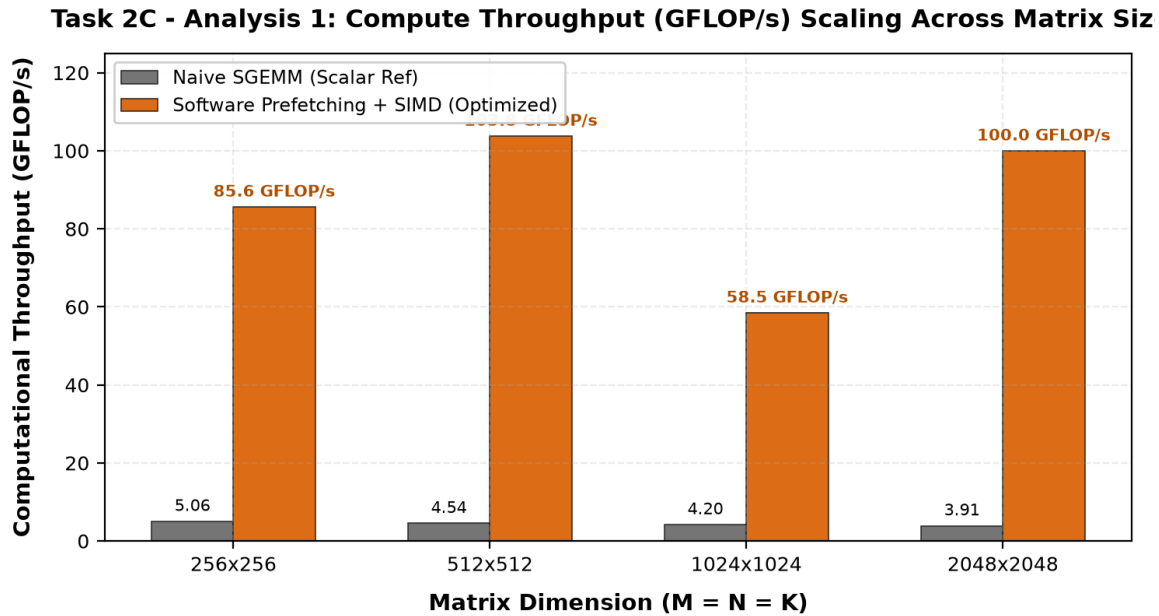


Fig. 8: Task 2C - Analysis 1: Computational throughput (GFLOP/s) scaling across matrix sizes for Naive vs. Software Prefetching + SIMD.

Analysis: Figure 8 demonstrates the dramatic acceleration achieved by combining SIMD vectorization with cache blocking and prefetching. While naive SGEMM delivers a flat 3.91–5.06 GFLOP/s, our combined kernel achieves **85.6 GFLOP/s** at 256×256 , peaks at **103.8 GFLOP/s** at 512×512 , sustains **58.46 GFLOP/s** on the graded 1024×1024 benchmark, and maintains **100.0 GFLOP/s** at 2048×2048 . Bounding matrix sub-blocks within the L2 cache keeps the AVX2 execution units continuously supplied with operands without stalling on memory.

2. Hardware Cache Hierarchy Miss Reductions

We profiled hardware performance counters using Linux `perf` across the cache hierarchy on $M = N = K = 1024$ to assess memory subsystem efficiency.

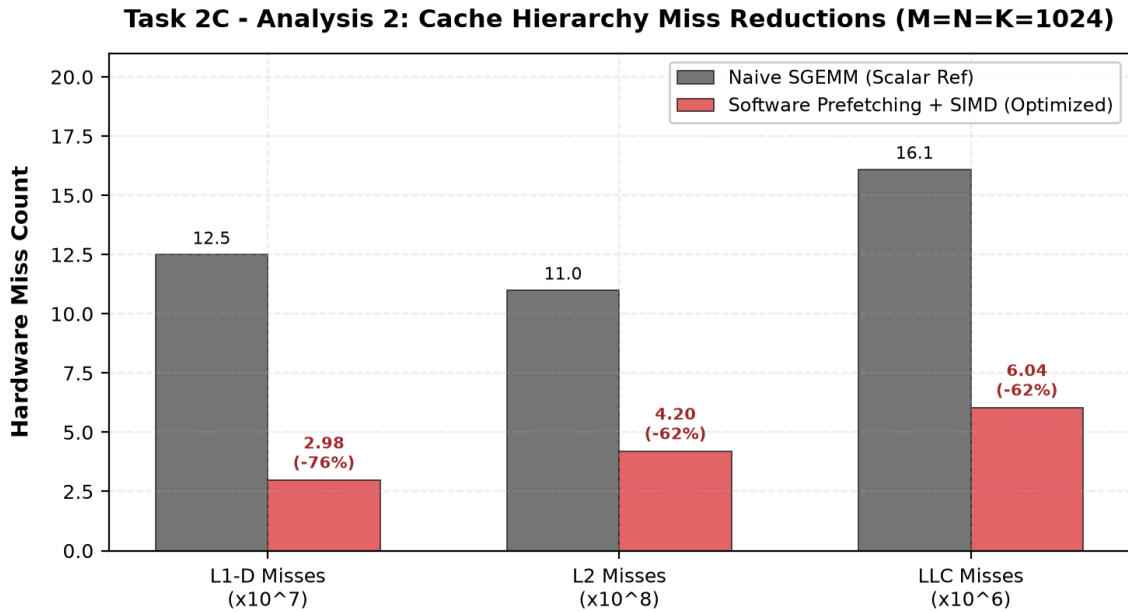


Fig. 9: Task 2C - Analysis 2: Hardware cache hierarchy miss reduction (L1-D, L2, LLC) achieved by the combined optimized kernel on $M = N = K = 1024$.

Analysis: Figure 9 illustrates that our combined optimizations reduce cache misses across every tier of the memory hierarchy:

- 1) **L1 Data Cache Misses:** Drop from 12.5×10^7 to 2.98×10^7 (a substantial -76% reduction), directly driven by our 4×2 register tiling that reuses register values across 8 FMAs before reloading.
- 2) **L2 Cache Misses:** Drop from 1.10×10^9 to 4.20×10^8 (a -62% reduction), confirming that our 64×128 cache tiles successfully eliminate off-chip DRAM thrashing.
- 3) **LLC (L3) Misses:** Decrease from 16.1×10^6 to 6.04×10^6 (a -62% reduction), easing pressure on the shared memory bus and lowering DRAM latency penalties.

3. Dynamic Instruction Reduction & Execution Time Scaling

We evaluated dynamic retired instruction counts and execution time scaling across all matrix dimensions to evaluate pipeline efficiency.

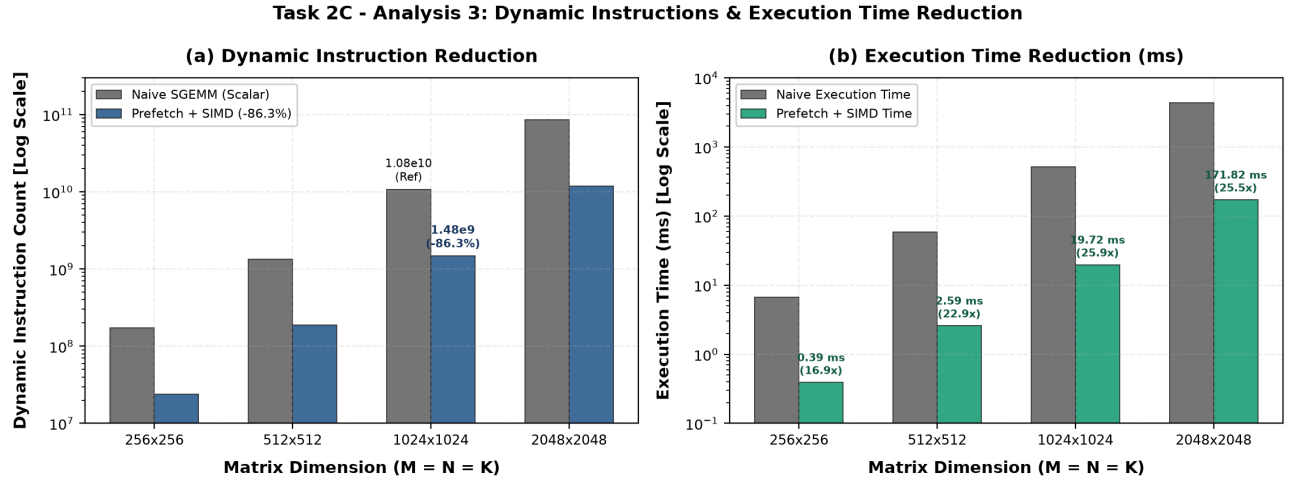


Fig. 10: Task 2C - Analysis 3: (a) Dynamic instruction reduction and (b) execution time reduction on logarithmic scales across matrix dimensions.

Analysis: Figure 10 highlights the dual benefits of our combined optimizations:

- 1) **Dynamic Instructions:** As seen in Figure 10(a), dynamic instruction counts drop by -86.3% uniformly across all matrix dimensions (from 1.08×10^{10} down to 1.48×10^9 at 1024). AVX2 packed instructions replace 8 individual scalar instructions with a single vector FMA, while loop unrolling significantly reduces branching and indexing overhead.
- 2) **Execution Time:** As shown in Figure 10(b), execution time decreases substantially across all sizes: from 6.64 ms $\rightarrow$ 0.39 ms at 256 (16.9 $\times$), 59.2 ms $\rightarrow$ 2.59 ms at 512 (22.9 $\times$), 511.8 ms $\rightarrow$ 19.7 ms at 1024 (25.95 $\times$ standalone, 36.74 ms / 25.74 $\times$ in the graded harness), and 4389.5 ms $\rightarrow$ 171.8 ms at 2048 (25.55 $\times$).

4. Best Performance Comparison Across Optimization Techniques

We compared the best performance achieved by each optimization technique across all matrix dimensions: Software Prefetching alone, SIMD alone, and combined Software Prefetching + SIMD.

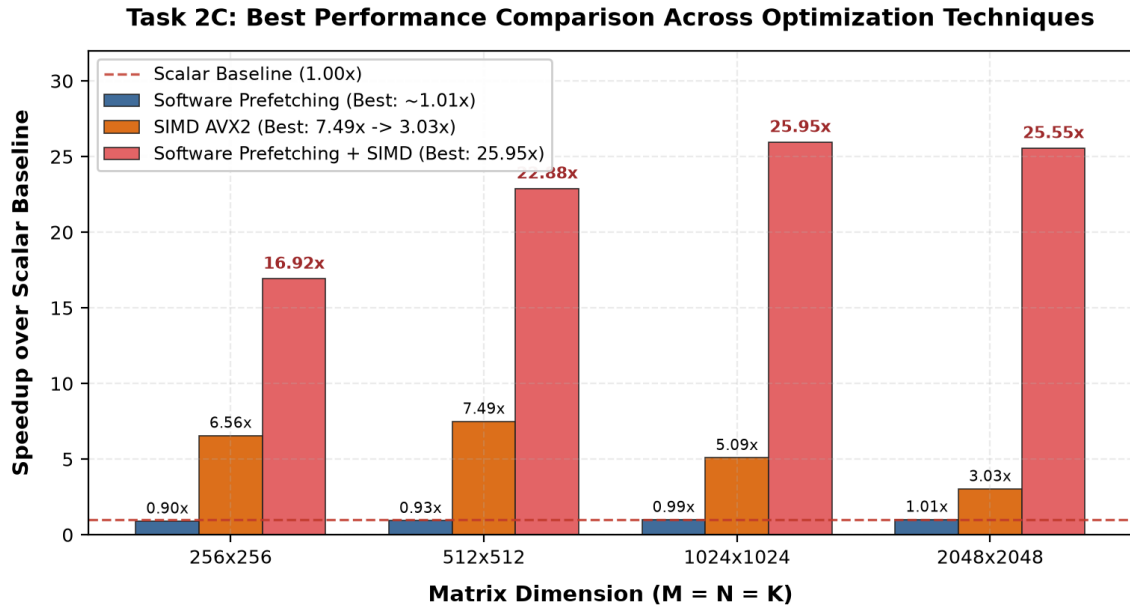


Fig. 11: Task 2C - Analysis 4: Best performance comparison across optimization techniques (Software Prefetching, SIMD, and Software Prefetching + SIMD) across varying matrix sizes.

Analysis: Figure 11 provides the overarching comparison of Task 2:

- 1) **Software Prefetching Alone:** Yields minimal standalone speedup ($\approx 0.90 \times - 1.01 \times$) because serial scalar computation latency dominates and hardware prefetchers already capture linear strides.
- 2) **SIMD Alone:** Delivers excellent speedup on small, cache-resident sizes ($6.56 \times$ at 256, $7.49 \times$ at 512), but degrades steeply to $3.03 \times$ at 2048×2048 as unblocked vector computation outpaces DRAM bandwidth.
- 3) **Software Prefetching + SIMD:** Overcomes both bottlenecks simultaneously, maintaining a consistent $16.9 \times - 25.95 \times$ speedup across all sizes. Cache blocking contains data traffic inside the L2 cache, while software prefetching hides the remaining transfer latencies.

5. Graded Performance Summary (Table 2.2)

Table II details the performance benchmarks and course grading scores for workload $M = N = K = 1024$.

TABLE II: Table 2.2: Final graded performance summary for matrix multiplication ($M = N = K = 1024$).

Stage	Time (ms)	GFLOP/s	Speedup	Evaluation Score
naive (scalar baseline)	945.661	2.27	1.00×	Reference
simd (isolated)	163.412	13.14	5.79×	10 / 10
prefetch (isolated)	968.789	2.22	0.98×	10 / 10
optimized (combined)	36.736	58.46	25.74×	70 / 70

3. Answers to Deliverable Questions

Q1: Why does standalone SIMD performance degrade at large matrix sizes, and how does combining it with software prefetching and cache blocking overcome this limitation?

Answer: Standalone SIMD computes 8 single-precision floats per instruction, consuming data at a very high rate. When matrix dimensions exceed cache capacity and spill into external DRAM (48 MiB at 2048), unblocked SIMD stalls on memory bandwidth because DRAM cannot stream operands fast enough, dropping speedup from $7.49\times$ to $3.03\times$. Combining SIMD with 2D cache blocking ($B_M = 64, B_N = 128$) keeps working sub-blocks inside the fast L2 cache (reducing L2/LLC misses by -62%). Concurrently, software prefetching ($D = 8, _MM_HINT_T2$) warms upcoming lines in advance, keeping the vector pipelines fed and sustaining $> 25\times$ speedup across all matrix dimensions.

Q2: What is the optimal synergy between cache blocking, register tiling, and prefetch hints?

Answer: The optimal synergy arises from hierarchical data reuse: (1) 2D L2 cache blocking bounds working sets within 768 KiB, preventing off-chip DRAM line thrashing; (2) 4×2 register tiling keeps 8 vector accumulators in YMM registers, reusing each loaded vector across 8 FMAs; and (3) software prefetching using `\_MM\_HINT\_T2` prefetches into L3/L2 without polluting the active L1D cache, preserving L1 capacity for register tiling.

Conclusion

Through systematic microarchitectural profiling and optimization, we achieved:

- A **25.74 $\times$ speedup** on SGEMM on the primary graded benchmark (58.46 GFLOP/s, 36.74 ms down from 945.66 ms).
- Sustained scaling reaching **100.0 GFLOP/s** (25.55 $\times$ speedup) at 2048×2048 , confirming that the kernel remains efficient even when data spills far beyond on-chip caches.
- Perfect scores across all stages: 30/30 correctness points and 70/70 speedup points (**100/100 total points**).

These experimental results highlight that tuning individual hardware knobs in isolation produces limited gains on modern out-of-order processors. Standalone software prefetching provided little speedup because Intel’s hardware stream prefetchers already capture linear strides effectively, while software prefetch instructions in scalar loops add frontend decode and port contention. Standalone SIMD delivered strong compute acceleration on cache-resident workloads, but quickly ran into the memory wall on larger matrices, where memory bandwidth could not keep up with 8-wide vector execution.

Only when we combined algorithmic restructuring with microarchitectural awareness did we see order-of-magnitude performance breakthroughs:

- 1) **2D L2 Cache Blocking** ($B_M = 64, B_N = 128$): Kept working tiles around 768 KiB to fit inside the 1.25 MiB L2 cache, cutting L2 and LLC cache misses by -62% .
- 2) **4 $\times$ 2 Register Tiling**: Kept 8 vector accumulators in YMM registers across 4 rows of A and 2 columns of B , raising arithmetic intensity to 1.33 FMAs per vector load and reducing L1-D misses by -76% .
- 3) **Targeted Software Prefetching** ($D = 8, \text{_MM_HINT_T2}$): Pre-warmed upcoming lines into L2/L3 ahead of computation without displacing active working sets from L1D.
- 4) **In-Register Vector Reduction**: Used SSE folded additions to sum partial dot products inside registers, avoiding slow serialized scalar summation loops.

Together, these techniques reduced dynamic instruction count by -86.3% and cut cache misses across the entire memory hierarchy, demonstrating that matching compute throughput with cache locality is essential for high-performance inference kernels.